

Spring Bean

In **Spring Boot** (and Spring Framework in general), a **bean** is basically an **object that is managed by the Spring IoC (Inversion of Control) container**. Instead of you manually creating objects with new, Spring creates, configures, and manages them for you.

Here's a clear breakdown:

1. Definition

A **Spring Bean** is:

- Any Java object that is instantiated, assembled, and managed by the Spring container.
 - Configured either using **annotations** or **XML** (though annotations are more common in Spring Boot).
-

2. How Beans are Created

In Spring Boot, beans can be created mainly in three ways:

a) Using **@Component** and **Stereotypes**

```
import org.springframework.stereotype.Component;
```

```
@Component
public class MyService {
    public void serve() {
        System.out.println("Service running...");
    }
}
```

- Spring automatically detects this class (via **component scanning**) and creates a bean for it.

Other stereotypes:

- **@Service** → for service layer
- **@Repository** → for DAO/repository layer

- @Controller / @RestController → for web layer
-

b) Using @Bean in @Configuration class

```
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;
```

```
@Configuration  
public class AppConfig {  
  
    @Bean  
    public MyService myService() {  
        return new MyService();  
    }  
}
```

- Here, Spring calls myService() and manages its lifecycle as a bean.
 - Useful when you cannot modify the class to add @Component.
-

c) Using @Service, @Repository, @Controller directly

- These are special kinds of @Component and are automatically detected by Spring Boot's **auto-scan**.
-

3. Bean Lifecycle

Spring manages the bean's lifecycle:

1. **Instantiation** → Spring creates the object.
 2. **Dependency Injection** → Spring injects required dependencies.
 3. **Initialization** → Any init methods run.
 4. **Ready to use**
 5. **Destruction** → When container closes, cleanup happens.
-

4. Dependency Injection with Beans

Once a bean exists, you can inject it anywhere:

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Controller;
```

```
@Controller  
public class MyController {  
  
    @Autowired  
    private MyService myService;  
  
    public void run() {  
        myService.serve();  
    }  
}
```

- @Autowired tells Spring: "inject the bean of type MyService here."

✓ **In short:** A **bean** is a Spring-managed object whose lifecycle and dependencies are handled automatically by the Spring IoC container.

I don't understand why there are so many ways to create a bean. It's confusing. Can you explain?

Ah! I get exactly why this feels confusing — Spring has **multiple ways to create beans**, but the good news is: you **don't need to use all of them at once**. Each way exists for slightly different scenarios. Let me simplify it for you.

Think of a bean as “any object Spring controls for you” .

And there are basically **two main ways you will create a bean in real life**:

1 Automatic Detection (Most Common in Spring Boot)

- You add `@Component` / `@Service` / `@Repository` to your class.
- Spring Boot **automatically scans your project** and creates the bean.

Example:

```
@Service
public class MyService {
    public void doSomething() {
        System.out.println("Doing work...");
    }
}
```

- Pros: Easy, no extra config.
 - Use this **99% of the time** for your own classes.
-

2 Manual Bean Creation (Special Cases)

- Sometimes the class is **from a library** or you **cannot modify it** to add `@Component`.
- Then you create a bean using `@Bean` **inside a `@Configuration` class**.

Example:

```
@Configuration
public class AppConfig {

    @Bean
    public ExternalService externalService() {
        return new ExternalService();
    }
}
```

- Pros: You control exactly how the object is created.
 - Use this for **third-party classes** or **special configuration**.
-

✓ Key Takeaway

- Most of the time: **just use `@Service` / `@Component`**, let Spring handle it.
 - Only use `@Bean` when you **can't put `@Component` on the class**.
-

Give a complete program implemented manually using multiple beans and a @RestController. I want a full student use case example. ?

Program 01:

Got it! Let's create a **simple Spring Boot project** demonstrating **manual bean creation** using @Bean in a @Configuration class. We'll keep it small and clear.

Project Structure

```
spring-boot-manual-bean/  
├─ src/main/java/com/example/demo/  
│   ├── DemoApplication.java  
│   ├── config/  
│   │   └─ AppConfig.java  
│   ├── service/  
│   │   └─ MessageService.java  
│   └─ controller/  
│       └─ MessageController.java  
└─ pom.xml
```

Step 1: Add a dependency in pom.xml

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-web</artifactId>  
  </dependency>  
</dependencies>
```

Step 2: Create the class you cannot annotate with @Component

```
// src/main/java/com/example/demo/service/MessageService.java
package com.example.demo.service;

public class MessageService {
    public String getMessage() {
        return "Hello from Manual Bean!";
    }
}
```

Step 3: Create a Configuration class to manually create the bean

```
// src/main/java/com/example/demo/config/AppConfig.java
package com.example.demo.config;

import com.example.demo.service.MessageService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public MessageService messageService() {
        return new MessageService(); // manually creating the bean
    }
}
```

Step 4: Use the bean in a controller

```
// src/main/java/com/example/demo/controller/MessageController.java
package com.example.demo.controller;

import com.example.demo.service.MessageService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MessageController {
```

```
private final MessageService messageService;

public MessageController(MessageService messageService) {
    this.messageService = messageService; // Spring injects the manual bean
}

@GetMapping("/message")
public String getMessage() {
    return messageService.getMessage();
}
}
```

Step 5: Main Spring Boot Application

```
// src/main/java/com/example/demo/DemoApplication.java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

✓ Run and Test

1. Start the application: `mvn spring-boot:run`
2. Open browser: `http://localhost:8080/message`
3. You should see:

Hello from Manual Bean!

This is a **classic example** of creating beans manually using @Bean when the class cannot be annotated with @Component.

If you want, I can also make a **version with 2 manual beans and one depending on the other**, so it shows **dependency injection between manual beans**.

Do you want me to do that?

Program 02 :

Perfect! Let' s extend the project to include **2 manual beans**, where one depends on the other. This demonstrates **manual dependency injection**.

Updated Project Structure

```
spring-boot-manual-bean/  
├─ src/main/java/com/example/demo/  
│   ├─ DemoApplication.java  
│   ├─ config/  
│   │   └─ AppConfig.java  
│   ├─ service/  
│   │   ├─ MessageService.java  
│   │   └─ GreetingService.java  
│   └─ controller/  
│       └─ MessageController.java  
└─ pom.xml
```

Step 1: Create the first bean (MessageService)

```
// src/main/java/com/example/demo/service/MessageService.java  
package com.example.demo.service;
```

```
public class MessageService {  
    public String getMessage() {  
        return "Hello from MessageService!";  
    }  
}
```

```
}  
}
```

Step 2: Create a second bean (GreetingService) that depends on MessageService

```
// src/main/java/com/example/demo/service/GreetingService.java  
package com.example.demo.service;
```

```
public class GreetingService {  
  
    private final MessageService messageService;  
  
    public GreetingService(MessageService messageService) {  
        this.messageService = messageService;  
    }  
  
    public String getGreeting() {  
        return "Greeting: " + messageService.getMessage();  
    }  
}
```

Step 3: Configuration class with both manual beans

```
// src/main/java/com/example/demo/config/AppConfig.java  
package com.example.demo.config;
```

```
import com.example.demo.service.MessageService;  
import com.example.demo.service.GreetingService;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;
```

```
@Configuration  
public class AppConfig {  
  
    @Bean  
    public MessageService messageService() {
```

```
        return new MessageService(); // first manual bean
    }

    @Bean
    public GreetingService greetingService(MessageService messageService) {
        return new GreetingService(messageService); // inject first bean into second
    }
}
```

Step 4: Controller uses GreetingService

```
// src/main/java/com/example/demo/controller/MessageController.java
package com.example.demo.controller;
```

```
import com.example.demo.service.GreetingService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MessageController {

    private final GreetingService greetingService;

    public MessageController(GreetingService greetingService) {
        this.greetingService = greetingService; // Spring injects the bean
    }

    @GetMapping("/greeting")
    public String getGreeting() {
        return greetingService.getGreeting();
    }
}
```

Step 5: Run the Application

1. Start: `mvn spring-boot:run`
2. Open browser: `http://localhost:8080/greeting`

You should see:

Greeting: Hello from MessageService!

✓ Key Points Shown Here:

1. Manual bean creation using @Bean.
 2. Bean dependency injection (GreetingService depends on MessageService).
 3. Spring automatically wires the dependencies.
-
-

Program 03 :

Perfect! Let's do a **complete Spring Boot program** using **manual bean creation** (@Bean in @Configuration) with **multiple beans**, a **REST controller**, and a **student use case**. I'll keep it clear and working end-to-end.

Use Case:

We have a **Student Management API**:

- List students
 - Add a student
 - Each student has **name, age, and course**
 - All beans are **manually created** (no @Component on classes)
-

1 Project Structure

src/main/java/com/example/studentapp

|

└─ Student.java

- |— StudentService.java
- |— StudentRepository.java
- |— AppConfig.java
- |— StudentController.java
- |— StudentAppApplication.java

2 Student Model

```
package com.example.studentapp;
```

```
public class Student {  
    private int id;  
    private String name;  
    private String course;  
  
    public Student(int id, String name, String course) {  
        this.id = id;  
        this.name = name;  
        this.course = course;  
    }  
  
    // Getters and Setters  
    public int getId() { return id; }  
    public String getName() { return name; }  
    public String getCourse() { return course; }  
    public void setId(int id) { this.id = id; }  
    public void setName(String name) { this.name = name; }  
    public void setCourse(String course) { this.course = course; }  
}
```

3 Repository Bean

```
package com.example.studentapp;
```

```
import java.util.ArrayList;  
import java.util.List;
```

```
public class StudentRepository {
```

```
private List<Student> students = new ArrayList<>();

public List<Student> findAll() {
    return students;
}

public void save(Student student) {
    students.add(student);
}
}
```

4 Service Bean

```
package com.example.studentapp;

import java.util.List;

public class StudentService {
    private final StudentRepository repository;

    // Constructor injection
    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    public void addStudent(Student student) {
        repository.save(student);
    }
}
```

5 Configuration Class (@Configuration)

```
package com.example.studentapp;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean
```

```
    public StudentRepository studentRepository() {
```

```
        return new StudentRepository();
```

```
    }
```

```
    @Bean
```

```
    public StudentService studentService(StudentRepository repository) {
```

```
        return new StudentService(repository);
```

```
    }
```

```
}
```

Notice: Spring **automatically injects** StudentRepository into StudentService constructor.

6 REST Controller

```
package com.example.studentapp;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;
```

```
@RestController
```

```
@RequestMapping("/students")
```

```
public class StudentController {
```

```
    private final StudentService service;
```

```
    public StudentController(StudentService service) {
```

```
        this.service = service;
```

```
    }
```

```
@GetMapping
public List<Student> getStudents() {
    return service.getAllStudents();
}

@PostMapping
public String addStudent(@RequestBody Student student) {
    service.addStudent(student);
    return "Student added successfully!";
}
}
```

7 Main Application

```
package com.example.studentapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudentAppApplication.class, args);
    }
}
```

8 Testing the API

- **Get all students:**

GET http://localhost:8080/students

- **Add a student:**

POST http://localhost:8080/students
Content-Type: application/json

{


```
"id": 1,  
"name": "Aniket",  
"course": "Java"  
}
```

✓ Highlights of Manual Bean Way:

1. All beans are created in AppConfig using @Bean.
 2. Dependencies are injected via **constructor** automatically by Spring.
 3. No @Component, @Service, or @Repository annotations are used on classes.
 4. Clean separation: Repository → Service → Controller.
-

ANIKET VARPE

Give a complete Spring Boot program using automatic detection (most common approach), with @RestController and multiple beans, for a full student use case."

Perfect! Let' s do a **full Spring Boot program** for a **Student use case** using **Automatic Detection** with @RestController and @Service. We' ll keep it **simple and complete**, so you can run it as-is.

Project Structure

student-demo

```
|
|— src/main/java/com/example/studentdemo
|   |— StudentDemoApplication.java    // Main Spring Boot app
|   |— controller/StudentController.java
|   |— service/StudentService.java
|   |— model/Student.java
|— pom.xml
```

1 pom.xml (Maven dependencies)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>student-demo</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.2</version>
  </parent>
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

2 Model Class: Student.java

```
package com.example.studentdemo.model;
```

```
public class Student {
    private int id;
    private String name;
    private int age;

    public Student() {}

    public Student(int id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
```

```
public void setName(String name) { this.name = name; }

public int getAge() { return age; }
public void setAge(int age) { this.age = age; }
}
```

3 Service Class: StudentService.java

```
package com.example.studentdemo.service;

import com.example.studentdemo.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service // <-- Automatic detection
public class StudentService {

    private List<Student> students = new ArrayList<>();

    public List<Student> getAllStudents() {
        return students;
    }

    public Student addStudent(Student student) {
        students.add(student);
        return student;
    }

    public Student getStudentById(int id) {
        return students.stream()
            .filter(s -> s.getId() == id)
            .findFirst()
            .orElse(null);
    }
}
```

4 Controller Class: StudentController.java

```
package com.example.studentdemo.controller;

import com.example.studentdemo.model.Student;
import com.example.studentdemo.service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired // Spring injects the bean automatically
    private StudentService studentService;

    // GET all students
    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    // GET student by ID
    @GetMapping("/{id}")
    public Student getStudent(@PathVariable int id) {
        return studentService.getStudentById(id);
    }

    // POST add student
    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return studentService.addStudent(student);
    }
}
```

5 Main Application Class: StudentDemoApplication.java

```
package com.example.studentdemo;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentDemoApplication.class, args);
    }
}
```

✓ How to Run

1. Run the main class StudentDemoApplication.
2. Open **Postman** or browser to test endpoints:

Method	URL	Body (if POST)	Action
GET	http://localhost:8080/students	—	List all students
GET	http://localhost:8080/students/1	—	Get student by ID
POST	http://localhost:8080/students	{ "id": 1, "name": "Aniket", "age": 25 }	Add new student

💡 Notes:

- @Service → Spring automatically detects it as a bean.
 - @RestController → Combines @Controller + @ResponseBody.
 - @Autowired → Injects the StudentService bean automatically.
-

Using @Component Annotation

Got it! Let's make a **simpler version using only annotations** (@Component + @Autowired) instead of manual @Bean creation. This is the most common Spring Boot style.

Project Structure

```
spring-boot-annotation-bean/
├── src/main/java/com/example/demo/
│   ├── DemoApplication.java
│   ├── service/
│   │   ├── MessageService.java
│   │   └── GreetingService.java
│   └── controller/
│       └── MessageController.java
└── pom.xml
```

Step 1: MessageService

```
// src/main/java/com/example/demo/service/MessageService.java
package com.example.demo.service;

import org.springframework.stereotype.Component;

@Component
public class MessageService {
    public String getMessage() {
        return "Hello from MessageService!";
    }
}
```

Step 2: GreetingService (depends on MessageService)

```
// src/main/java/com/example/demo/service/GreetingService.java
package com.example.demo.service;

import org.springframework.stereotype.Component;

@Component
public class GreetingService {

    private final MessageService messageService;

    public GreetingService(MessageService messageService) { // constructor
injection
        this.messageService = messageService;
    }

    public String getGreeting() {
        return "Greeting: " + messageService.getMessage();
    }
}
```

```
}  
}
```

Step 3: Controller

```
// src/main/java/com/example/demo/controller/MessageController.java  
package com.example.demo.controller;  
  
import com.example.demo.service.GreetingService;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;  
  
@RestController  
public class MessageController {  
  
    private final GreetingService greetingService;  
  
    public MessageController(GreetingService greetingService) {  
        this.greetingService = greetingService;  
    }  
  
    @GetMapping("/greeting")  
    public String getGreeting() {  
        return greetingService.getGreeting();  
    }  
}
```

Step 4: Main Application

```
// src/main/java/com/example/demo/DemoApplication.java  
package com.example.demo;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class DemoApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(DemoApplication.class, args);  
    }  
}
```

Step 5: Run and Test

1. Start app: `mvn spring-boot:run`
2. Visit: `http://localhost:8080/greeting`
3. Output:

Greeting: Hello from MessageService!

✓ **Key Points:**

- `@Component` automatically creates beans.
- Spring injects dependencies automatically via constructor.
- No need for `@Configuration` and `@Bean` if the class can be annotated.

If you want, I can **also make a side-by-side comparison** of **manual `@Bean` vs annotation `@Component`** style—it's very helpful for learning.

Do you want me to make that comparison?

Production-ready Spring Boot app – More practice code..

Got it! Let's take your **manual bean approach** and upgrade it to a **production-ready Spring Boot app** for a **Student Management System**. I'll include:

- Proper **layer separation**: Controller → Service → Repository
- **DTOs** for request/response
- **Exception handling**
- **Validation**
- **Logging**
- **Manual beans only** (`@Bean` in `@Configuration`)
- **Multiple services** for demonstration

1 Project Structure

src/main/java/com/example/studentapp

|

|— controller

| |— StudentController.java

```
├── service
|   ├── StudentService.java
|   └── CourseService.java
├── repository
|   └── StudentRepository.java
├── dto
|   └── StudentDTO.java
├── exception
|   ├── StudentNotFoundException.java
|   └── GlobalExceptionHandler.java
├── config
|   └── AppConfig.java
└── StudentAppApplication.java
```

2 DTO (Request/Response Object)

```
package com.example.studentapp.dto;
```

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
```

```
public class StudentDTO {
```

```
    @NotNull(message = "ID is required")
    private Integer id;
```

```
    @NotBlank(message = "Name is required")
    private String name;
```

```
    @NotBlank(message = "Course is required")
    private String course;
```

```
// Getters & Setters
```

```
public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getCourse() { return course; }
public void setCourse(String course) { this.course = course; }
}
```

3 Repository

```
package com.example.studentapp.repository;

import com.example.studentapp.dto.StudentDTO;

import java.util.*;

public class StudentRepository {

    private final Map<Integer, StudentDTO> studentDb = new HashMap<>();

    public List<StudentDTO> findAll() {
        return new ArrayList<>(studentDb.values());
    }

    public StudentDTO findById(int id) {
        return studentDb.get(id);
    }

    public void save(StudentDTO student) {
        studentDb.put(student.getId(), student);
    }

    public void deleteById(int id) {
        studentDb.remove(id);
    }

    public boolean existsById(int id) {
        return studentDb.containsKey(id);
    }
}
```

4 Service Layer

StudentService.java

```
package com.example.studentapp.service;
```

```

import com.example.studentapp.dto.StudentDTO;
import com.example.studentapp.exception.StudentNotFoundException;
import com.example.studentapp.repository.StudentRepository;

import java.util.List;

public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public List<StudentDTO> getAllStudents() {
        return repository.findAll();
    }

    public StudentDTO getStudentById(int id) {
        StudentDTO student = repository.findById(id);
        if (student == null) throw new StudentNotFoundException("Student not found with ID: "
+ id);
        return student;
    }

    public void addStudent(StudentDTO student) {
        repository.save(student);
    }

    public void deleteStudent(int id) {
        if (!repository.existsById(id))
            throw new StudentNotFoundException("Cannot delete. Student not found with ID: " +
id);
        repository.deleteById(id);
    }
}

```

CourseService.java (another bean for demonstration)

```
package com.example.studentapp.service;

import java.util.List;
import java.util.Arrays;

public class CourseService {

    public List<String> getAvailableCourses() {
        return Arrays.asList("Java", "Spring Boot", "React", "Python");
    }
}
```

5 Controller Layer

```
package com.example.studentapp.controller;

import com.example.studentapp.dto.StudentDTO;
import com.example.studentapp.service.StudentService;
import com.example.studentapp.service.CourseService;

import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService studentService;
    private final CourseService courseService;

    public StudentController(StudentService studentService, CourseService courseService) {
        this.studentService = studentService;
        this.courseService = courseService;
    }

    @GetMapping
    public List<StudentDTO> getAllStudents() {
        return studentService.getAllStudents();
    }
}
```

```

    @GetMapping("/{id}")
    public StudentDTO getStudent(@PathVariable int id) {
        return studentService.getStudentById(id);
    }

    @PostMapping
    public String addStudent(@Valid @RequestBody StudentDTO student) {
        studentService.addStudent(student);
        return "Student added successfully!";
    }

    @DeleteMapping("/{id}")
    public String deleteStudent(@PathVariable int id) {
        studentService.deleteStudent(id);
        return "Student deleted successfully!";
    }

    @GetMapping("/courses")
    public List<String> getCourses() {
        return courseService.getAvailableCourses();
    }
}

```

6 Exception Handling

StudentNotFoundException.java

```

package com.example.studentapp.exception;

public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
}

```

GlobalExceptionHandler.java

```

package com.example.studentapp.exception;

```

```

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpStatus;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<String> handleNotFound(StudentNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
    handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach((error) -> {
            String fieldName = ((FieldError) error).getField();
            String message = error.getDefaultMessage();
            errors.put(fieldName, message);
        });
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
    }
}

```

7 Configuration (@Bean manual)

```

package com.example.studentapp.config;

import com.example.studentapp.repository.StudentRepository;
import com.example.studentapp.service.StudentService;
import com.example.studentapp.service.CourseService;

import org.springframework.context.annotation.Bean;

```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean
```

```
    public StudentRepository studentRepository() {
```

```
        return new StudentRepository();
```

```
    }
```

```
    @Bean
```

```
    public StudentService studentService(StudentRepository repository) {
```

```
        return new StudentService(repository);
```

```
    }
```

```
    @Bean
```

```
    public CourseService courseService() {
```

```
        return new CourseService();
```

```
    }
```

```
}
```

8 Main Application

```
package com.example.studentapp;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class StudentAppApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(StudentAppApplication.class, args);
```

```
    }
```

```
}
```

9 Features / Production-ready improvements

- **Validation** for requests (@Valid)

- **Global exception handling** with proper HTTP status codes
- **Multiple manual beans** (StudentService, CourseService, StudentRepository)
- **Clean layering**
- **REST endpoints** with proper URI design

Endpoints:

HTTP	URI	Action
GET	/api/students	List all students
GET	/api/students/{id}	Get student by ID
POST	/api/students	Add student
DELETE	/api/students/{id}	Delete student
GET	/api/students/courses	List all courses

If you want, I can **also add logging, swagger docs, and in-memory H2 DB integration** to make it fully production-grade like a **real enterprise app**.

Do you want me to do that next?